OpenMPI
Simulación transferencia de calor

Computación de altas prestaciones

Máster universitario ingeniería informática



VNiVERSiDAD D SALAMANCA

Autores

Lur Perona Bzik

Alejandro Martín Martín

Fecha

Noviembre de 2025

Profesor

Javier Blanco

Tabla de contenidos

1 Introducción **1**

2 Descripción de la solución **1**

3 Detalles de implementación **4**

4 Entorno de pruebas **6**

5 Resultados de las pruebas **7**

6 Rendimiento **7**

6.1 Speedup 7

6.2 Eficiencia 7

7 Comparación de rendimiento frente PThreads **8**

7.1 Conclusiones 8

Apéndice **10**

A Tablas de resultados OpenMPI **10**

B Tablas de resultados PThreads **11**

Referencias **13**

Listado de figuras

1 Comunicación entre procesos vecinos. 2
2 Comunicación global entre procesos. 3

Listado de tablas

1 Tiempos medios (en segundos) según número de procesos y tamaño de la matriz . 7
2 Speedup respecto a la ejecución secuencial (1 proceso). 7
3 Eficiencia del algoritmo para distintos tamaños de matriz. 7
4 Comparativa de tiempos MPI/Pthreads para 2 Unidades (procesos/hilos) 8
5 Comparativa de tiempos MPI/Pthreads para 4 Unidades (procesos/hilos) 8
6 Comparativa de tiempos MPI/Pthreads para 8 Unidades (procesos/hilos) 8
7 Comparativa de tiempos MPI/Pthreads para 16 Unidades (procesos/hilos) 8
8 Tiempos de ejecución para 1 procesos 10
9 Tiempos de ejecución para 2 procesos 10
10 Tiempos de ejecución para 4 procesos 10
11 Tiempos de ejecución para 8 procesos 10
12 Tiempos de ejecución para 16 procesos 11
13 Tiempos de ejecución para 20 procesos 11
14 Tiempos de ejecución para 1 hilo 11
15 Tiempos de ejecución para 2 hilos 11
16 Tiempos de ejecución para 4 hilos 12
17 Tiempos de ejecución para 8 hilos 12
18 Tiempos de ejecución para 16 hilos 12
19 Tiempos de ejecución para 24 hilos 12

Listado de ejemplos de código

1 Inicialización de MPI 4
2 Registro de un nuevo tipo de datos 4
3 Distribución de tareas 4
4 Creación de topología cartesiana 1D 4
5 Recepción y envío entre vecinos 5
6 Patrón de comunicación de MPI_Neighbor_alltoallw 5
7 Envío, cálculo y recepción del diferencial de temperatura 6
8 Recepción de resultados en el proceso raíz 6
9 Finalización de MPI 6

Listado de transformaciones

1 Distribución de una matriz global en submatrices. 1
2 Conversión de submatriz global a local. 1
3 Búferes globales y locales de envío y recepción. 2
4 Conversión de submatriz ampliada global a local. 2

1. Introducción

En este documento se explica la realización de una simulación de transferencia de calor realizada utilizando la librería MPI así como un análisis de los resultados obtenidos y una comparación con la versión realizada en PThreads.

2. Descripción de la solución

Hemos visto anteriormente como resolver este problema de computación empleando múltiples hilos que operan sobre una matriz en memoria compartida. Para esta nueva solución se utilizan diferentes procesos y, por tanto, se hará uso de memoria distribuida. La división de trabajo para cada proceso será equivalente a la versión realizada con hilos, cada proceso se encargará de computar un bloque de filas determinado, tal y como se muestra en la Transformación 1.

$$G = \left[\begin{array}{cccc} g_{11} & g_{12} & \cdots & g_{1n} \\ g_{21} & g_{22} & \cdots & g_{2n} \\ g_{31} & g_{32} & \cdots & g_{3n} \\ \hline g_{41} & g_{42} & \cdots & g_{4n} \\ g_{51} & g_{52} & \cdots & g_{5n} \\ g_{61} & g_{62} & \cdots & g_{6n} \\ \hline g_{71} & g_{72} & \cdots & g_{7n} \\ g_{81} & g_{82} & \cdots & g_{8n} \\ g_{91} & g_{92} & \cdots & g_{9n} \end{array}\right] \qquad G^{(p1)} = \left[\begin{array}{cccc} g_{11} & g_{12} & \cdots & g_{1n} \\ g_{21} & g_{22} & \cdots & g_{2n} \\ g_{31} & g_{32} & \cdots & g_{3n} \end{array}\right]$$

$$G^{(p2)} = \left[\begin{array}{cccc} g_{41} & g_{42} & \cdots & g_{4n} \\ g_{51} & g_{52} & \cdots & g_{5n} \\ g_{61} & g_{62} & \cdots & g_{6n} \end{array}\right]$$

$$G^{(p3)} = \left[\begin{array}{cccc} g_{71} & g_{72} & \cdots & g_{7n} \\ g_{81} & g_{82} & \cdots & g_{8n} \\ g_{91} & g_{92} & \cdots & g_{9n} \end{array}\right]$$

Transformación 1: Distribución de una matriz global en submatrices.

La principal diferencia es que cada proceso no tendrá en memoria la matriz completa sino tan sólo las filas que se le hayan asignado.

$$G^{(i)} = \begin{bmatrix} g_{r,1} & \cdots & g_{r,n} \\ g_{r+1,1} & \cdots & g_{r+1,n} \\ \vdots & \ddots & \vdots \\ g_{s,1} & \cdots & g_{s,n} \end{bmatrix} \qquad L^{(i)} = \begin{bmatrix} l_{1,1} & \cdots & l_{1,n} \\ l_{2,1} & \cdots & l_{2,n} \\ \vdots & \ddots & \vdots \\ l_{s-r+1,1} & \cdots & l_{s-r+1,n} \end{bmatrix}$$

Transformación 2: Conversión de submatriz global a local.

De esta forma, tal y como se muestra en la Transformación 2, los procesos únicamente tendrán acceso a su matriz local y no serán conscientes de las posiciones globales sobre las que operan.
Al realizar esta división de trabajo, cada proceso requiere conocer los valores situados en la fila superior e inferior globales con respecto a su matriz local. Por tanto, será necesario establecer comunicaciones entre cada par de procesos vecinos, tal y como muestra la Figura 1.

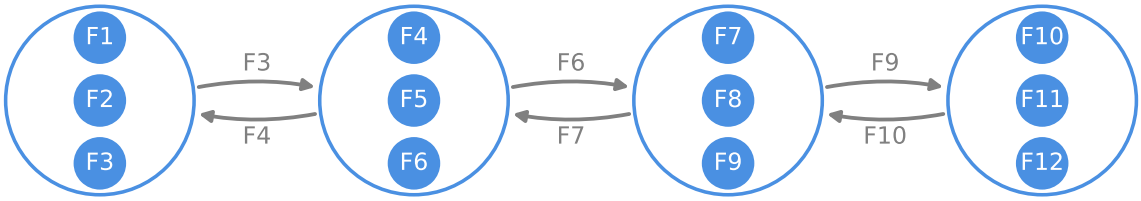


Figura 1: Comunicación entre procesos vecinos.

Teniendo esto en cuenta, cada proceso necesitara un total de **cuatro búferes** de comunicación de mensajes, dos de envío y dos de recepción.

$$BG^{(i)}_{\text{envío}} = \begin{bmatrix} g_{r,1} & \cdots & g_{r,n} \\ g_{s,1} & \cdots & g_{s,n} \end{bmatrix} \qquad BL^{(i)}_{\text{envío}} = \begin{bmatrix} l_{1,1} & \cdots & l_{1,n} \\ l_{s-r+1,1} & \cdots & l_{s-r+1,n} \end{bmatrix}$$

$$BG^{(i)}_{\text{recepción}} = \begin{bmatrix} g_{r-1,1} & \cdots & g_{r-1,n} \\ g_{s+1,1} & \cdots & g_{s+1,n} \end{bmatrix} \qquad BL^{(i)}_{\text{recepción}} = \begin{bmatrix} m_{1,1} & \cdots & m_{1,n} \\ m_{2,1} & \cdots & m_{2,n} \end{bmatrix}$$

Transformación 3: Búferes globales y locales de envío y recepción.

En la Transformación 3 se muestra el contenido de los búferes de envío y recepción donde cada proceso:

- **enviará** su primera y última fila local.
- **recibirá** la fila anterior y posterior global a su matriz local.

De esta forma, una vez establecidas las comunicaciones entre procesos, cada proceso tendrá en memoria la matriz ampliada que permite realizar los cómputos para cada uno de los valores en su matriz local en el siguiente instante de tiempo k:

$$G^{+(i)} = \left[\begin{array}{ccc} g_{r-1,1} & \cdots & g_{r-1,n} \\ \hline g_{r,1} & \cdots & g_{r,n} \\ g_{r+1,1} & \cdots & g_{r+1,n} \\ \vdots & \ddots & \vdots \\ g_{s,1} & \cdots & g_{s,n} \\ \hline g_{s+1,1} & \cdots & g_{s+1,n} \end{array}\right] \qquad L^{+(i)} = \left[\begin{array}{ccc} m_{1,1} & \cdots & m_{1,n} \\ \hline l_{1,1} & \cdots & l_{1,n} \\ l_{2,1} & \cdots & l_{2,n} \\ \vdots & \ddots & \vdots \\ l_{s-r+1,1} & \cdots & l_{s-r+1,n} \\ \hline m_{2,1} & \cdots & m_{2,n} \end{array}\right]$$

Transformación 4: Conversión de submatriz ampliada global a local.

Además de las comunicaciones descritas hasta el momento, será necesario un intercambio de mensajes adicional para determinar la condición de fin del cómputo. Esta condición (de la misma forma que la versión realizada con hilos) es:

$$\sum_{i=1}^{n}\sum_{j=1}^{m} |\text{MatrizActual}_{i,j} - \text{MatrizAnterior}_{i,j}| < 10^{-6}$$

Es decir, cuando se alcance un valor estacionario.

En la figura 2 se muestra un diagrama para un caso concreto (12 filas y 4 procesos) de todas las comunicaciones necesarias entre procesos en dos instantes de tiempo diferentes.

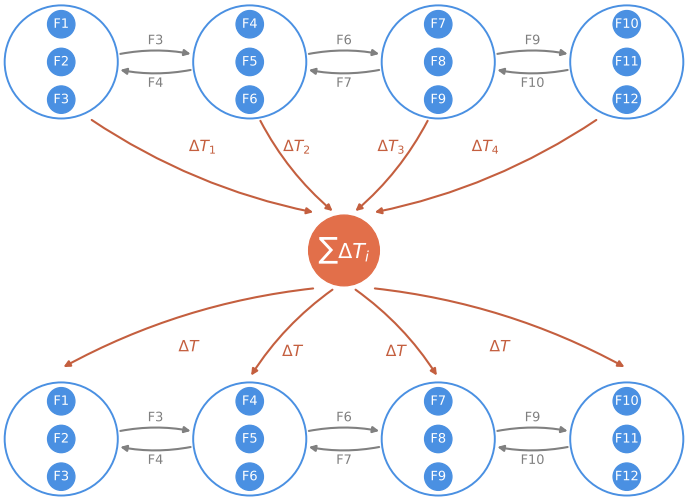


Figura 2: Comunicación global entre procesos.

El orden temporal de las acciones realizadas en un instante de tiempo k por cada proceso sería el siguiente:

- k_1. Envía y recibe mensajes con sus vecinos.
- k_2. Realiza los cálculos sobre su matriz local.
- k_3. Envía el diferencial de temperatura local obtenido.
- k_4. Recibe el diferencial de temperatura obtenido y comprueba la condición de fin.
- $(k+1)_1$. Envía y recibe mensajes con sus vecinos.

  ⋮

El último punto a comentar respecto a la solución propuesta es la asignación de tareas a cada proceso. Por lo general, cuando trabajamos con computación paralela en memoria distribuida —especialemente en problemas de estas características—, es altamente recomendable disponer de los datos de forma completa en un único proceso antes y después (nunca durante) de los cómputos realizados.
Una forma de lograrlo es asignar un proceso raíz encargado de:

- Reservar memoria e inicializar los valores de la matriz global.
- Distribuir entre los procesos las filas que les correspondan.
- Recibir los resultados enviados por cada proceso una vez hayan finalizado sus cálculos.

De esta forma modificar el estado inicial del problema es trivial y facilita la interpretación de los resultados obtenidos.
Como una posible alternativa —más adecuada en aquellos casos donde el tamaño del problema sea muy grande—, podría inicializarse los datos en cada proceso de forma individual y gestionar la escritura de resultados de forma ordenada. De esta forma, se evita cargar el problema entero en la memoria de un proceso.

3. Detalles de implementación

En esta sección se abordan los detalles de implementación realizados en el código haciendo uso de MPI. Se detalla únicamete las funciones de MPI empleadas para resolver el problema descrito en la sección anterior.

```
1    MPI_Init(&argc, &argv);
2    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
3    MPI_Comm_rank(MPI_COMM_WORLD, &id);
```

Código 1: Inicialización de MPI

```
1    MPI_Type_contiguous(global.ncol, MPI_FLOAT, &send_root.tipo_fila);
2    MPI_Type_commit(&send_root.tipo_fila);
```

Código 2: Registro de un nuevo tipo de datos

Se registra un tipo de datos *tipo_fila* para definir el tamaño de los mensajes enviados y recibidos tanto en la lógica relacionada con la distribución de tareas como la comunicación entre vecinos. La razón de utilizar este tipo derivado es simplificar la definición del tamaño de los mensajes ya que en ambos casos son múltiplos del tamaño de una fila.

```
1    MPI_Scatterv(
2        id == 0 ? global.datos  : NULL,
3        id == 0 ? send_root.contadores_env  : NULL,
4        id == 0 ? send_root.despls          : NULL,
5        id == 0 ? send_root.tipo_fila       : MPI_DATATYPE_NULL,
6        local.datos_anterior,
7        local.nfil,
8        send_root.tipo_fila,
9        0,
10       MPI_COMM_WORLD
11   );
```

Código 3: Distribución de tareas

Para realizar la distribución de tareas se utiliza *MPI_Scatterv*. El proceso raíz distribuye la matriz global entre todos los procesos —incluyéndose a sí mismo—.

```
1    topologia.dims[0] = nprocs;
2    topologia.periods[0] = 0;
3    MPI_Comm comm1D;
4    MPI_Cart_create(
5        MPI_COMM_WORLD,
6        1,
7        topologia.dims,
8        topologia.periods,
9        0,
10       &comm1D
11   );
```

Código 4: Creación de topología cartesiana 1D

Se define una topología cartesiana de una dimensión. Es equivalente a la topología lineal con la diferencia de que MPI conoce internamente cuáles son los vecinos de cada proceso en ese

comunicador. Esto se hace para simplificar la comunicación entre vecinos, como se verá más adelante.

```
MPI_Neighbor_alltoallw(
    local.matriz_anterior[0],
    send_recv_count,
    send_despl,
    send_recv_type,
    recv_buf,
    send_recv_count,
    recv_despl,
    send_recv_type,
    comm1D
);
```

Código 5: Recepción y envío entre vecinos

MPI_Neighbor_alltoallw nos permite enviar a cada uno de nuestros vecinos un mensaje diferente. Para emplear esta función es necesario haber definido una topología previamente. A continuación se muestra el flujo de funcionamiento de esta función:

```
    MPI_Cart_get(comm, maxdims, dims, periods, coords);
    for (dim = 0, i = 0 ; dim < dims ; ++dim) {
        MPI_Cart_shift(comm, dim, 1, &r0, &r1);
        MPI_Isend(sendbuf + sdispls[i]  * extent(sendtype),
                  sendcount, sendtypes[i], r0, ..., comm, ...);
        MPI_Irecv(recvbuf + rdispls[i] * extent(recvtype),
                  recvcount, recvtypes[i], r0, ..., comm, ...);
        ++i;
        MPI_Isend(sendbuf + sdispls[i] * extent(sendtype),
                  sendcount, sendtypes[i], r1, ..., comm, &req[i]);
        MPI_Irecv(recvbuf + rdispls[i] * extent(recvtype),
                  recvcount, recvtypes[i], r1, ..., comm, ...);
        ++i;
    }
    MPI_Wait_all (...);
```

Código 6: Patrón de comunicación de MPI_Neighbor_alltoallw
Fuente: (The Open MPI Project, 2019)

Se itera sobre las dimensiones definidas en la topología asociada al comunicador, se obtienen los vecinos en cada dimensión y se realizan envíos y recepciones no bloqueantes. Por último, se espera a que se completen todas las llamadas no bloqueantes, es decir, finalicen los envíos y recepciones realizados.

La ventaja de este enfoque frente a realizar llamadas bloqueantes es que no se fuerza un orden determinado en los envíos y recepciones. De esta forma, se mejora el rendimiento y se reduce la dificultad a la hora de evitar interbloqueos.

```
    MPI_Allreduce(
        &local_dt,
        &global_dt,
        1,
        MPI_FLOAT,
        MPI_SUM,
        MPI_COMM_WORLD
    );
```

Código 7: Envío, cálculo y recepción del diferencial de temperatura

Se envía el diferencial de temperatura local de cada proceso, se calcula la suma global y se devuelve el resultado a cada proceso. Se emplea para verificar la condición de finalización. Nótese que esta función también actúa como barrera de sincronización, puesto que un proceso no puede continuar hasta que todos hayan enviado su valor local y se haya completado la operación.

```
    MPI_Gatherv(
        local.datos_actual,
        local.nfil,
        send_root.tipo_fila,
        id == 0 ? global.datos : NULL,
        id == 0 ? send_root.contadores_env : NULL,
        id == 0 ? send_root.despls : NULL,
        id == 0 ? send_root.tipo_fila : MPI_DATATYPE_NULL,
        0,
        MPI_COMM_WORLD
    );
```

Código 8: Recepción de resultados en el proceso raíz

Una vez finalizados los cálculos, cada proceso envía sus datos locales al proceso raíz.

```
    MPI_Finalize();
```

Código 9: Finalización de MPI

4. Entorno de pruebas

Para la realización de las pruebas experimentales y posterior análisis de los resultados obtenidos, se ha empleado el servidor de prácticas **vdia3.fis.usal.es**, cuyas características son las siguientes ("Servidor vdía 3", 2025):

- Dell PowerEdge R440
- 2 x Xeon Silver 4210R (20 Cores-40 Threads)
- 128GB de RAM DDR4-3200. (4x32GB Hynix HMA84GR7CJR4N-XN)
- SSD 512GB – RAID 1
- Sistema Linux Debian 10 – 64bits

Por tanto, al contar con 20 núcleos físicos, el límite empleado en la realización de las pruebas experimentales será de 20 procesos —frente a los 40 hilos empleados para la ejecución del programa que emplea Pthreads—.

5. Resultados de las pruebas

En esta sección se evalúan los resultados obtenidos de las ejecuciones con diferentes tamaños de matriz y números de hilos. En todos los casos se han considerado matrices cuadradas. Para maximizar la fiabilidad de los resultados obtenidos, cada caso se ha ejecutado 5 veces y se trabajará con la media temporal obtenida. Pueden verse los resultados desglosados en el apéndice A.

Procesos	**N=100**	**N=500**	**N=750**	**N=1000**
1	3.4104	81.0702	181.5155	324.2330
2	1.9436	45.0008	101.1649	179.5952
4	1.0412	22.5876	50.9449	90.1492
8	0.6213	11.8705	26.4777	46.7820
16	0.4244	6.1477	13.3603	23.7190
20	0.5133	8.4323	18.9014	33.6105

Tabla 1: Tiempos medios (en segundos) según número de procesos y tamaño de la matriz

6. Rendimiento

En esta sección se analizan métricas de rendimiento en base a los resultados obtenidos.

6.1. Speedup

La expresión del speedup es $S = \frac{T_{\text{secuencial}}}{T_{\text{paralelo}}}$.

En la tabla 2 se muestra el speedup para múltiples números de procesos y tamaños de matrices.

Procesos	**N=100**	**N=500**	**N=750**	**N=1000**
2	1.7547	1.8015	1.7943	1.8054
4	3.2755	3.5891	3.5630	3.5966
8	5.4891	6.8296	6.8554	6.9307
16	8.0358	13.1871	13.5862	13.6698
20	6.6441	9.6142	9.6033	9.6468

Tabla 2: Speedup respecto a la ejecución secuencial (1 proceso).

En los resultados de la tabla anterior vemos que no hay ningún valor donde $S < P$, no hay speedup superlineal.

6.2. Eficiencia

La eficiencia se define como $E = \frac{S}{p} = \frac{T_{\text{secuencial}}}{p \cdot T_{\text{paralelo}}}$.

Procesos	**N = 100**	**N = 500**	**N = 750**	**N = 1000**
2	0.8773	0.9008	0.8971	0.9027
4	0.8189	0.8973	0.8907	0.8992
8	0.6861	0.8537	0.8569	0.8663
16	0.5022	0.8242	0.8491	0.8544
20	0.3322	0.4807	0.4802	0.4823

Tabla 3: Eficiencia del algoritmo para distintos tamaños de matriz.

7. Comparación de rendimiento frente PThreads

En esta sección se emplean los resultados anteriores presentados en este informe y los resultados obtenidos en la práctica anterior (también disponibles en el apéndice B), con el fin de comparar el rendimiento obtenido en cada una de las versiones.
El primer punto que es preciso comentar es el procedimiento empleado para comprobar que ambos programas resuelven el mismo algoritmo. Para ello hemos comprobado que el resultado final es el mismo, es decir, los valores de la matriz obtenidos en el estado estacionario son iguales y, por otra parte, que el número de iteraciones temporales computadas hasta alcanzar el estado estacionario es equivalente.

N	Tiempo MPI (s)	Tiempo PThreads(s)	Diferencia relativa (%)
100	1.9436	1.8176	-6.48
500	45.0008	40.2785	-10.49
750	101.1649	90.1637	-10.87
1000	179.5952	158.7615	-11.60

Tabla 4: Comparativa de tiempos MPI/Pthreads para 2 Unidades (procesos/hilos)

N	Tiempo MPI (s)	Tiempo PThreads(s)	Diferencia relativa (%)
100	1.0412	1.0624	2.04
500	22.5876	20.9299	-7.34
750	50.9449	46.9997	-7.74
1000	90.1492	83.3607	-7.53

Tabla 5: Comparativa de tiempos MPI/Pthreads para 4 Unidades (procesos/hilos)

N	Tiempo MPI (s)	Tiempo PThreads(s)	Diferencia relativa (%)
100	0.6213	0.8278	33.24
500	11.8705	11.0775	-6.68
750	26.4777	24.3756	-7.94
1000	46.7820	43.3023	-7.44

Tabla 6: Comparativa de tiempos MPI/Pthreads para 8 Unidades (procesos/hilos)

N	Tiempo MPI (s)	Tiempo PThreads(s)	Diferencia relativa (%)
100	0.4244	1.1966	181.9
500	6.1477	7.1092	15.6
750	13.3603	13.8142	3.40
1000	23.7190	23.0468	-2.84

Tabla 7: Comparativa de tiempos MPI/Pthreads para 16 Unidades (procesos/hilos)

7.1. Conclusiones

Analizando los resultados obtenidos, podemos comprobar que el rendimiento es superior en Pthreads con respecto a MPI. Esta brecha aumenta conforme lo hace el tamaño del problema con respecto al número de unidades de ejecución (hilos/procesos).
Este es el resultado esperado, principalmente por el siguiente motivo:
La penalización en el rendimiento en la versión multi-hilo, se debe únicamente a los mecanimos de sincronización, mientras que la versión multi-proceso —además de tener mecanismos de sincronización como *Waitall*— requiere de paso de mensajes. El paso de mensajes implica copias de datos tanto

en el envío como recepción que penalizan el rendimiento, siendo este el principal factor influyente. Además, hay que tener en cuenta otros factores como los cambios de contexto en el planificador de tareas, los cuáles son más ligeros y rápidos entre hilos que entre procesos.

Apéndice

A. Tablas de resultados OpenMPI

Nprocs	Nfil	Ncol	Tiempos	Media
1	100	100	3.411303 3.414885 3.412173 3.398166 3.415651	3.4104
1	500	500	81.152463 81.166187 80.944176 80.988550 81.099559	81.0702
1	750	750	181.167505 181.630953 181.319863 181.251177 182.207988	181.5155
1	1000	1000	325.265384 325.445903 322.270554 323.201327 324.982000	324.2330

Tabla 8: Tiempos de ejecución para 1 procesos

Nprocs	Nfil	Ncol	Tiempos	Media
2	100	100	1.951654 1.936983 1.942897 1.944980 1.941481	1.9436
2	500	500	44.989948 44.998448 44.988829 45.025007 45.001529	45.0008
2	750	750	101.165855 101.142739 101.091531 101.070837 101.353544	101.1649
2	1000	1000	179.492380 179.817597 179.284818 179.728393 179.652856	179.5952

Tabla 9: Tiempos de ejecución para 2 procesos

Nprocs	Nfil	Ncol	Tiempos	Media
4	100	100	1.044984 1.030678 1.059837 1.043003 1.027562	1.0412
4	500	500	22.572755 22.617122 22.586743 22.575690 22.585534	22.5876
4	750	750	51.344520 50.755694 50.932452 50.843063 50.848992	50.9449
4	1000	1000	90.015414 90.382427 90.176396 90.055470 90.116043	90.1492

Tabla 10: Tiempos de ejecución para 4 procesos

Nprocs	Nfil	Ncol	Tiempos	Media
8	100	100	0.613472 0.643121 0.604078 0.630825 0.615038	0.6213
8	500	500	12.140598 11.847613 11.794545 11.768857 11.800651	11.8705
8	750	750	26.450747 26.997151 26.260048 26.305535 26.374984	26.4777
8	1000	1000	46.702168 46.612345 47.320426 46.580565 46.694535	46.7820

Tabla 11: Tiempos de ejecución para 8 procesos

Nprocs	Nfil	Ncol	Tiempos	Media
16	100	100	0.415499 0.448085 0.418914 0.421116 0.418399	0.4244
16	500	500	6.110775 6.230804 6.159282 6.117338 6.120086	6.1477
16	750	750	13.378487 13.341870 13.338021 13.322667 13.420518	13.3603
16	1000	1000	23.713870 23.714590 23.729737 23.596980 23.839957	23.7190

Tabla 12: Tiempos de ejecución para 16 procesos

Nprocs	Nfil	Ncol	Tiempos	Media
20	100	100	0.505958 0.514392 0.515262 0.511793 0.518949	0.5133
20	500	500	8.429775 8.411843 8.440219 8.433065 8.446400	8.4323
20	750	750	18.913220 18.893453 18.906508 18.895971 18.897701	18.9014
20	1000	1000	33.583850 33.357371 33.584310 33.916290 33.607182	33.6105

Tabla 13: Tiempos de ejecución para 20 procesos

B. Tablas de resultados PThreads

Nhilos	Nfil	Ncol	Tiempos	Media
1	100	100	3.411303 3.414885 3.412173 3.398166 3.415651	3.4104
1	500	500	81.152463 81.166187 80.944176 80.988550 81.099559	81.0702
1	750	750	181.167505 181.630953 181.319863 181.251177 182.207988	181.5155
1	1000	1000	325.265384 325.445903 322.270554 323.201327 324.982000	324.2330

Tabla 14: Tiempos de ejecución para 1 hilo

Nhilos	Nfil	Ncol	Tiempos	Media
2	100	100	1.810529 1.815243 1.808209 1.832405 1.821855	1.8176
2	500	500	40.505589 40.391796 40.618285 39.305677 40.571045	40.2785
2	750	750	90.269440 90.110582 90.317294 89.850673 90.270464	90.1637
2	1000	1000	157.370708 159.897996 158.341329 157.918840 160.278606	158.7615

Tabla 15: Tiempos de ejecución para 2 hilos

Nhilos	Nfil	Ncol	Tiempos	Media
4	100	100	1.062461 1.068199 1.039712 1.067426 1.074141	1.0624
4	500	500	21.101317 20.678984 21.175523 21.263932 20.429946	20.9299
4	750	750	47.020373 47.024512 46.891735 47.059177 47.002877	46.9997
4	1000	1000	83.463848 83.087294 83.359679 83.460219 83.432621	83.3607

Tabla 16: Tiempos de ejecución para 4 hilos

Nhilos	Nfil	Ncol	Tiempos	Media
8	100	100	0.858636 0.840474 0.840920 0.769323 0.829728	0.8278
8	500	500	11.084514 11.050140 11.070920 11.099064 11.082725	11.0775
8	750	750	24.183463 24.475747 24.411507 24.379662 24.427841	24.3756
8	1000	1000	43.262399 43.720478 43.231268 43.123253 43.173873	43.3023

Tabla 17: Tiempos de ejecución para 8 hilos

Nhilos	Nfil	Ncol	Tiempos	Media
16	100	100	1.192517 1.182777 1.239566 1.178387 1.189842	1.1966
16	500	500	7.464415 7.123108 6.927613 7.019921 7.011021	7.1092
16	750	750	13.945058 13.779426 13.622320 13.725844 13.998459	13.8142
16	1000	1000	23.062797 23.046139 23.163797 23.005058 22.956177	23.0468

Tabla 18: Tiempos de ejecución para 16 hilos

Nhilos	Nfil	Ncol	Tiempos	Media
24	100	100	1.847302 1.850927 1.853913 1.851644 1.897905	1.8603
24	500	500	7.909710 7.841926 7.916852 7.870033 7.885448	7.8848
24	750	750	16.065069 15.948341 16.128525 16.101858 15.918161	16.0324
24	1000	1000	27.304458 27.333808 27.468352 27.629601 27.763750	27.5000

Tabla 19: Tiempos de ejecución para 24 hilos

Referencias

Servidor vdía 3 [Disponible en línea.]. (2025). https://studium25.usal.es/pluginfile.php/416402/mod_resource/content/4/servidor_vdia3.pdf

The Open MPI Project. (2019). *MPI_Neighbor_alltoallw — All processes send data of different types to, and receive data of different types from, all processes* [Man page version 2.1.6, last modified 20-May-2019]. Open MPI. https://www.open-mpi.org/doc/v2.1/man3/MPI_Neighbor_alltoallw.3.php